

Taller – Frontend para la Heladería con React + Vite y Supabase

Objetivo general

El objetivo de este taller es que los estudiantes implementen una aplicación frontend en React (con Vite) que consuma la API automática de Supabase, para gestionar los productos e ingredientes de una heladería, practicar operaciones CRUD, autenticación, autorización, manejo de inventario y despliegue.

Enunciado

Una heladería muy popular está buscando digitalizar su menú y parte de su manejo de inventario. Para esto ha decidido contar con nuestra ayuda.

La heladería tiene dos conceptos principales: Productos e Ingredientes. A partir de los ingredientes, que pueden ser bases o complementos, es posible armar productos, que pueden ser Copas o Malteadas.

Cada Ingrediente tiene: nombre, precio, número de calorías por porción, inventario disponible, indicador de si es o no vegetariano, indicador de si es sano según regulaciones del departamento de salud, tipo (base o complemento). Las bases tienen un sabor, y los complementos pueden bajar su inventario a 0 con una sola instrucción.

Los Productos están compuestos por 3 ingredientes, pueden ser copas (con tipo de vaso) o malteadas (con volumen en onzas), tienen nombre y precio al público y debe poder calcularse su costo, calorías y rentabilidad.

La heladería quiere saber cuál es el producto más rentable, vender productos revisando si hay ingredientes disponibles, y llevar un contador de las ventas del día.

Requisitos mínimos del proyecto

1. Frontend en React con Vite.
2. Estilos con TailwindCSS o Bootstrap.
3. Conexión a la API REST generada automáticamente por Supabase.
4. CRUD completo de ingredientes.
5. Listado de productos con ingredientes, mostrando calorías, costo y rentabilidad.
6. Función de venta de productos, que descuente inventario y registre ventas.
7. Autenticación de usuarios (login con Supabase Auth).
8. Autorización por roles:
 - a. Administrador → acceso a todas las funciones.

- b. Empleado → acceso a todas las funciones excepto las relacionadas con rentabilidad.
 - c. Cliente → acceso solo a la venta de productos, lista de productos y sus calorías.
 - d. Público (no autenticado) → solo puede ver la lista de productos.
9. La aplicación debe ser visualmente atractiva, incluir un logo, un nombre creativo para la heladería y mantener una interfaz clara y responsiva.
10. Se debe desplegar la aplicación en Vercel.

Operaciones obligatorias en la API

Usuarios

- Consultar usuario por correo y contraseña

Productos

- Consultar todos los productos
- Consultar un producto según su ID
- Consultar un producto según su nombre
- Consultar las calorías de un producto según su ID
- Consultar la rentabilidad de un producto según su ID
- Consultar el costo de producción de un producto según su ID
- Vender un producto según su ID

Ingredientes

- Consultar todos los ingredientes
- Consultar un ingrediente según su ID
- Consultar un ingrediente según su nombre
- Consultar si un ingrediente es sano según su ID
- Reabastecer un ingrediente según su ID
- Renovar el inventario de un ingrediente (poner a 0 si es complemento)

Guía de configuración de Supabase

1. Crear proyecto

Ingresa a <https://supabase.com>, crea un proyecto nuevo y accede a la **sección SQL Editor**.

2. Script SQL para la base de datos

Pegar el siguiente script en el editor y ejecutarlo:

```
-- Usuarios
create table public.users (
  id bigserial primary key,
```

```
nombre text not null,  
correo text unique not null,  
password text not null,  
rol text check (rol in ('admin', 'empleado', 'cliente')) not null default 'cliente'  
);
```

-- Ingredientes

```
create table public.ingredientes (  
  id bigserial primary key,  
  nombre text not null,  
  precio numeric(10,2) not null,  
  calorías integer not null,  
  inventario integer not null default 0,  
  es_vegetariano boolean not null default false,  
  es_sano boolean not null default true,  
  tipo text check (tipo in ('base', 'complemento')) not null,  
  sabor text  
);
```

-- Productos

```
create table public.productos (  
  id bigserial primary key,  
  nombre text not null,  
  precio_publico numeric(10,2) not null,  
  tipo text check (tipo in ('copa', 'malteada')) not null,  
  vaso text,  
  volumen_onzas integer  
);
```

-- Relación productos - ingredientes

```
create table public.producto_ingrediente (  
  id bigserial primary key,  
  producto_id bigint not null references public.productos(id) on delete cascade,  
  ingrediente_id bigint not null references public.ingredientes(id) on delete cascade  
);
```

-- Ventas

```
create table public.ventas (  
  id bigserial primary key,  
  producto_id bigint not null references public.productos(id),  
  user_id uuid references public.users(id),  
  fecha timestamp with time zone default now(),
```

```
cantidad integer not null default 1,  
total numeric(10,2) not null  
);
```

-- Vistas

```
create view public.v_calorias_producto as  
select p.id as producto_id, p.nombre,  
       sum(i.calorias) as total_calorias  
from public.productos p  
join public.producto_ingrediente pi on p.id = pi.producto_id  
join public.ingredientes i on pi.ingrediente_id = i.id  
group by p.id, p.nombre;
```

```
create view public.v_costo_producto as  
select p.id as producto_id, p.nombre,  
       sum(i.precio) as costo  
from public.productos p  
join public.producto_ingrediente pi on p.id = pi.producto_id  
join public.ingredientes i on pi.ingrediente_id = i.id  
group by p.id, p.nombre;
```

```
create view public.v_rentabilidad_producto as  
select p.id as producto_id, p.nombre,  
       p.precio_publico,  
       c.costo,  
       (p.precio_publico - c.costo) as rentabilidad  
from public.productos p  
join public.v_costo_producto c on p.id = c.producto_id;
```

3. Datos iniciales

Ejecutar este script para cargar datos de prueba:

```
insert into public.ingredientes  
(nombre, precio, calorias, inventario, es_vegetariano, es_sano, tipo, sabor)  
values  
(('Vainilla', 1000, 150, 50, true, true, 'base', 'vainilla'),  
(('Fresa', 1200, 100, 40, true, true, 'base', 'fresa'),  
(('Chocolate', 1500, 200, 30, true, true, 'base', 'chocolate'),  
(('Chispas de Chocolate', 500, 50, 25, true, false, 'complemento', null),  
(('Crema Batida', 300, 70, 20, true, false, 'complemento', null),  
(('Sirope de Fresa', 400, 60, 15, true, false, 'complemento', null),  
(('Oreo', 800, 110, 25, true, false, 'complemento', null);
```

```
insert into public.productos
```

```
(nombre, precio_publico, tipo, vaso, volumen_onzas)
```

```
values
```

```
('Copa Vainilla Deluxe', 5000, 'copa', 'mediano', null),
```

```
('Copa ChocoFresa', 6000, 'copa', 'grande', null),
```

```
('Malteada de Fresa', 7000, 'malteada', null, 16),
```

```
('Malteada Oreo', 7500, 'malteada', null, 20);
```

```
insert into public.producto_ingrediente (producto_id, ingrediente_id)
```

```
values
```

```
(1, 1), (1, 5), (1, 4),
```

```
(2, 3), (2, 2), (2, 6),
```

```
(3, 2), (3, 1), (3, 5),
```

```
(4, 3), (4, 7), (4, 5);
```

```
insert into public.ventas (producto_id, cantidad, total)
```

```
values
```

```
(1, 2, 10000),
```

```
(2, 1, 6000),
```

```
(3, 3, 21000),
```

```
(4, 1, 7500);
```

```
insert into public.users (nombre, correo, contra, rol)
```

```
values
```

```
('admin', 'admin@admin.co', 'admin', 'admin'),
```

```
('empleado', 'empleado@empleado.co', 'empleado', 'empleado'),
```

```
('cliente', 'cliente@cliente.co', 'cliente', 'cliente');
```

Entrega esperada

- Proyecto React + Vite conectado a Supabase.
- CRUD de ingredientes.
- Listado de productos con ingredientes.
- Visualización de calorías, costo y rentabilidad.
- Función de venta de productos (con actualización de inventario).
- Login de usuario
- Autorización basada en roles (admin, empleado, cliente, público).
- Página visualmente atractiva, con logo y nombre creativo para la heladería.
- Código subido a GitHub.
- Proyecto desplegado en Vercel.